INFRAESTRUCTURES PER AL BIG DATA
(17244106, 17254118, 17924106)
HW2: Analítica de Big Data con PySpark

- La fecha límite de entrega de este ejercicio es el **10 de mayo de 2026**.
- Se debe entregar **un archivo ZIP** con el código fuente y un informe en PDF.
  El código puede entregarse como **notebooks de Jupyter/Colab y/o archivos .py**, siempre que esté claramente organizado y sea ejecutable.
- La entrega del código y de la documentación debe realizarse a través de Moodle.

Descripción de la tarea: Nueva York genera diariamente enormes volúmenes de datos de movilidad urbana. En este proyecto, diseñaréis e implementaréis una tubería completa de analítica de big data con PySpark para procesar, limpiar, analizar y modelar datos de trayectos de taxi del conjunto NYC TLC Trip Record Data.

A diferencia de un ejercicio básico de Python, este proyecto se centra en conceptos de ingeniería y analítica de big data, incluyendo:

- Carga distribuida de datos y preprocesamiento con Spark
- Trabajo con grandes conjuntos de datos públicos en formato Parquet y/o CSV
- Limpieza y validación de datos a escala
- Ingeniería de características con PySpark
- Análisis exploratorio de datos con Spark SQL y DataFrames
- Optimización del rendimiento en Spark
- Agregaciones y analítica basada en ventanas
- Una tarea final avanzada con Spark MLlib o análisis de anomalías

El objetivo no es solo escribir código en PySpark, sino comprender cómo Spark procesa grandes volúmenes de datos, cómo las transformaciones distribuidas difieren del procesamiento local y cómo las decisiones de ingeniería afectan a la escalabilidad y al rendimiento.

Objetivos de aprendizaje: Al completar este proyecto, seréis capaces de:

- Cargar y procesar un gran conjunto de datos público utilizando PySpark.
- Aplicar limpieza e ingeniería de características en entornos distribuidos.
- Usar Spark DataFrames y Spark SQL para análisis a gran escala.
- Diseñar preguntas analíticas significativas sobre movilidad urbana.
- Aplicar técnicas de rendimiento como repartitioning, caching e inspección del plan de ejecución.
- Usar funciones de ventana para análisis avanzados.
- Implementar una tarea avanzada con Spark MLlib o detección de anomalías.
- Comunicar los hallazgos de manera clara en un informe técnico.

Estructura del equipo: Cada grupo debe estar formado por 3 estudiantes y asignar los siguientes roles:

- Data Engineer: responsable de la ingestión de datos, inspección del esquema, limpieza y almacenamiento.
- Analytics Engineer: responsable de Spark SQL, agregaciones, preguntas de negocio y resúmenes visuales.
- ML/Performance Engineer: responsable de la preparación de variables, la tarea avanzada, la optimización y el análisis de ejecución.

El informe final debe indicar claramente la contribución de cada miembro.

Entorno de trabajo: Los grupos pueden utilizar Google Colab Free Edition para este ejercicio. Dado que la edición gratuita tiene memoria y sesiones limitadas, se recomienda trabajar con un subconjunto moderado del dataset NYC TLC, por ejemplo 2–3 meses de registros, y justificar esta selección en el informe.

Descripción del conjunto de datos: Debéis utilizar NYC TLC Trip Record Data como conjunto principal. El análisis puede enriquecerse con la tabla de zonas de taxi cuando resulte útil. La carga mínima recomendada es de al menos 3 ficheros mensuales. Los grupos más fuertes pueden ampliar el análisis a 6 o más meses si su entorno lo permite.

Los atributos habituales disponibles incluyen fecha y hora de recogida, fecha y hora de llegada, identificador de zona de origen, identificador de zona de destino, número de pasajeros, distancia del trayecto, importe de la tarifa, tipo de pago e importe total.

Los grupos deben descargar el conjunto de datos desde la página oficial de **NYC TLC Trip Record Data**. Deben utilizar los archivos **Yellow Taxi Trip Records (PARQUET)** y la **Taxi Zone Lookup Table**. Para la parte obligatoria de la práctica, cada grupo debe descargar **3 archivos mensuales de Yellow Taxi** del mismo año, además de la tabla de consulta. Los estudiantes que deseen ampliar el análisis podrán utilizar más meses, siempre que su entorno de ejecución lo permita. La página oficial de TLC proporciona los archivos mensuales en formato Parquet y la documentación del conjunto de datos.

Ejemplo: Esta selección es simple, coherente y suficiente para un proyecto de Spark de 3 semanas. El sitio oficial de TLC proporciona archivos mensuales de **Yellow Taxi** en formato Parquet, y el conjunto de datos **Taxi Zones** es la tabla de referencia estándar para los identificadores de ubicación de recogida y destino utilizados en los registros de viajes de TLC.

- `yellow_tripdata_2024-01.parquet`
- `yellow_tripdata_2024-02.parquet`
- `yellow_tripdata_2024-03.parquet`
- `taxi_zone_lookup.csv`

<u>Pasos para completar la práctica:</u>

1. Adquisición y carga de datos

- Descargar los ficheros seleccionados de NYC TLC.
- Cargarlos en PySpark usando SparkSession.
- Leer los datos con el formato correcto, preferiblemente Parquet.
- Inspeccionar el esquema y los tipos de datos.
- Unificar varios ficheros mensuales en un único Spark DataFrame.

2. Limpieza y validación de datos

- Identificar y tratar registros problemáticos, como valores nulos en campos clave.
- Eliminar o justificar registros con distancia, duración o tarifa cero o negativa.
- Inspeccionar cantidades irreales de pasajeros y tiempos inconsistentes de recogida/llegada.
- Explicar y justificar todas las decisiones de limpieza en el informe.

3. Ingeniería de características

- Crear la duración del trayecto en minutos.
- Crear hora de recogida, día de la semana y mes.
- Calcular la velocidad media.
- Crear indicadores de eficiencia de tarifa, como tarifa por kilómetro.
- Añadir cualquier otra característica derivada útil y justificarla.

4. Análisis exploratorio con Spark Utilizad Spark DataFrames y/o Spark SQL para responder, como mínimo, a las siguientes cuestiones:

- ¿Cuáles son las horas con más recogidas?
- ¿Cuáles son las zonas de recogida y de destino más activas?
- ¿Cómo varía la demanda por día de la semana?
- ¿Qué zonas generan más ingresos totales?
- ¿Cuál es la distancia media de trayecto por hora?
- ¿Cuál es la tarifa media por día de la semana?
- ¿Qué tipo de pago es el más frecuente?
- ¿Qué zonas presentan la mayor duración media de viaje?
- ¿Qué zonas tienen trayectos cortos pero muy frecuentes?
- Identificad trayectos con velocidad media irreal.
- Identificad registros con tarifa sospechosamente alta para una distancia corta.
- Clasificad las 10 zonas de recogida principales por mes.
- Comparad el comportamiento de los trayectos de día frente a noche.
- Encontrad patrones de concentración de demanda a lo largo del tiempo.

5. Uso de funciones de ventana Debéis implementar al menos dos análisis con funciones de ventana, por ejemplo:

- Ranking de las principales zonas de recogida por día o por mes.
- Ranking de zonas con mayor ingreso por período.
- Cálculo de medias móviles del número de trayectos.
- Comparación de cada zona con la media de la ciudad.

6. Rendimiento y optimización Para al menos tres operaciones o consultas, haced lo siguiente:

- Ejecutad el análisis normalmente.
- Inspeccionad el plan de ejecución con explain().
- Aplicad una estrategia de optimización, como cache(), repartition(), poda de columnas o filtrado antes del join.
- Volved a ejecutar el análisis.
- Explicad qué mejoró y por qué.

7. Tarea avanzada Cada grupo debe completar una de las siguientes opciones:

- **Opción A** – Predicción con Spark MLlib: predecir duración del trayecto, importe de la tarifa o categoría de tarifa.
- **Opción B** – Clustering: agrupar zonas o trayectos según su comportamiento de movilidad.

- **Opción C** – Detección de anomalías: detectar trayectos sospechosos o anómalos mediante reglas o modelos justificados.

8. Breve reflexión En el informe, incluid una breve reflexión (8–12 líneas) sobre:

- Qué paso fue el más costoso computacionalmente y por qué.
- Qué optimización de Spark fue la más útil.
- Qué dificultades encontrasteis al trabajar con datos grandes en modo distribuido.
- Qué conclusiones revela el dataset sobre la movilidad urbana.

Instrucciones de entrega

Cada grupo debe entregar un único archivo ZIP que incluya:

1. Código fuente

- Un notebook o archivo .py para carga y limpieza.
- Un notebook o archivo .py para las consultas analíticas.
- Un notebook o archivo .py para la tarea avanzada.
- Comentarios claros y secciones bien organizadas.
- Nombre recomendado: HW2_NYC_Taxi_GroupXX.zip

2. Informe en PDF El informe debe incluir:

- Resumen del proyecto, objetivo, meses seleccionados y miembros del grupo.
- Comprensión de los datos: esquema, tamaño y atributos clave.
- Limpieza de datos: registros inválidos encontrados y reglas aplicadas.
- Ingeniería de características: variables creadas y justificación.
- Resultados de consultas y figuras con interpretación breve.
- Al menos dos análisis basados en funciones de ventana.
- Análisis de rendimiento para tres operaciones antes y después de la optimización.
- Metodología, resultados e interpretación de la tarea avanzada.
- Contribución de cada miembro.

Plataforma de entrega: Entregad el archivo ZIP a través de Moodle.

Criterios de evaluación

- Corrección de la carga y limpieza de datos: 20%
- Calidad de la ingeniería de características: 15%
- Calidad de las consultas analíticas y de los hallazgos: 25%
- Uso de funciones de ventana y herramientas avanzadas de Spark: 15%
- Análisis de rendimiento y optimización: 15%
- Claridad del informe, completitud y trabajo en equipo: 10%

Notas generales:

- No utilicéis Pandas para procesar el dataset completo.
- Usad Pandas solo para pequeñas tablas agregadas finales si lo necesitáis para representar gráficamente.
- La mayor parte del trabajo debe realizarse en PySpark.
- Utilizad comentarios significativos y una organización limpia del código.
- Comprobad que los resultados tengan sentido antes de escribir conclusiones.
- Cada miembro del grupo debe entender toda la tubería, no solo su parte asignada.

Uso de herramientas de IA generativa:

- El uso de herramientas de IA generativa (por ejemplo, ChatGPT, Copilot, etc.) está permitido solo como apoyo al aprendizaje.

- Debéis entender y ser capaces de explicar todo el código entregado.
- Está estrictamente prohibido copiar código generado por IA sin comprenderlo.
- Durante la evaluación o revisión oral, cualquier miembro del grupo podrá ser preguntado por cualquier parte de la entrega.
- Si utilizáis herramientas de IA, debéis indicarlo brevemente en el informe explicando para qué se usaron.
- Se aplican las normas de integridad académica. Las entregas que demuestren falta de comprensión podrán recibir una calificación reducida.

Guía para los estudiantes

1. Trabajad como un equipo real

- Dividid responsabilidades, pero aseguraos de que todos entienden el flujo completo.
- Llevad control del periodo de datos elegido, las reglas de limpieza, las variables creadas, las preguntas analíticas y la tarea avanzada.

2. Empezad pronto

- Semana 1: elegir el periodo de datos, cargar los datos, inspeccionar el esquema, limpiar registros y crear variables.
- Semana 2: responder las preguntas analíticas, construir figuras, implementar funciones de ventana y comenzar el análisis de rendimiento.
- Semana 3: completar la tarea avanzada, mejorar el código, redactar el informe y revisar el ZIP final.

3. Mantened el proyecto organizado

- Usad una estructura clara de carpetas, por ejemplo data/, notebooks/, scripts/, figures/ y report/.

4. Justificad cada decisión

- Explicad por qué se eliminaron filas, por qué una variable es útil, por qué se seleccionó un modelo y por qué una optimización ayudó.

5. Haced figuras legibles

- Todas las gráficas deben tener título, ejes etiquetados y leyendas legibles cuando sea necesario.

6. Centraos en Spark

- La lógica principal debe usar Spark DataFrames, Spark SQL, funciones de PySpark y Spark MLlib cuando corresponda.

7. Preparaos para posibles preguntas orales

- Debéis poder explicar por qué Spark es mejor que Pandas local en este caso, cómo se diferencian las transformaciones y acciones, y qué significan vuestras optimizaciones o modelos.